

BANK MANAGEMENT SYSTEM

Full System Documentation

Prepared by: Abdalla Firin

Final Year Project — University

Date: 9 May 2026

Laravel 12

Vue.js 3

Tailwind CSS

MySQL

Table of Contents

1. System Overview	3
2. Technology Stack	4
3. System Architecture	5
4. Database Design	6
5. Roles & Permissions (RBAC)	9
6. Multi-Branch Architecture	13
7. General Ledger (GL) System	15
8. Transaction & Approval Workflow	18
9. API Endpoints Reference	20
10. Security Design	22
11. Build Roadmap	23

1. System Overview

The Bank Management System (BMS) is a full-stack web application designed to manage the complete operations of a multi-branch bank. It provides secure, role-based access to three categories of users: System Administrators, Bank Staff, and Customers.

1.1 Core Objectives

- Provide a secure, role-based platform for all bank staff operations
- Implement a double-entry General Ledger ensuring financial integrity
- Support multiple branches with centralized data and isolated operations
- Enforce an atomic transaction model preventing money loss or duplication
- Maintain a complete, immutable audit trail of all system activities
- Provide customers with a self-service portal for account management

1.2 System Portals

Portal	Users	Purpose
Admin Portal	Super Admin, System Administrator	System-wide configuration, branch management, global reporting
Staff Dashboard	Branch Manager, Tellers, Officers, Auditors	Day-to-day banking operations, transactions, approvals
Customer Portal	Bank Customers	Self-service: balance, history, transfers, loan applications

2. Technology Stack

Layer	Technology	Version	Purpose
Backend	Laravel	12.x	REST API, business logic, RBAC, DB transactions
Frontend	Vue.js	3.x (Composition API)	SPA portals, reactive UI, role-aware components
Styling	Tailwind CSS	3.x	Utility-first professional UI design
Database	MySQL	8.x	Relational data, ACID transactions, foreign keys
Authentication	Laravel Sanctum	Built-in	API token auth for Vue SPA
State Management	Pinia	2.x	Auth store, permissions, branch state
HTTP Client	Axios	Latest	Vue-to-Laravel API communication
ORM	Eloquent	Built-in	Models, relationships, query builder
Version Control	Git + GitHub	Latest	Source control, commit history
API Testing	Postman	Latest	Endpoint testing and documentation

2.1 Why This Stack

Laravel 12 provides `DB::transaction()` for atomic operations, Sanctum for secure token auth, Eloquent ORM for clean database access, and Gates/Policies for RBAC — covering every banking requirement out of the box.

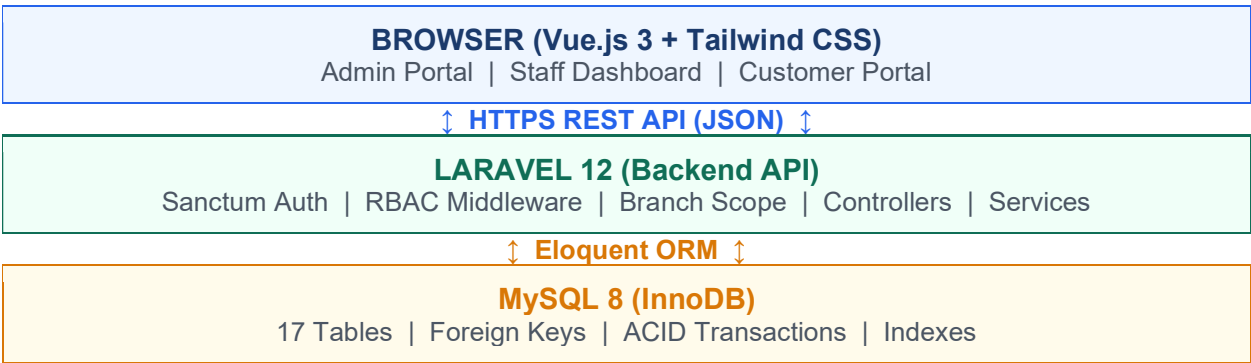
Vue.js 3 with the Composition API allows building three separate portals sharing reusable components. Pinia stores the authenticated user's permissions array, enabling real-time UI changes based on role. Navigation guards block unauthorized route access before any component loads.

MySQL with InnoDB engine is mandatory for banking — it is the only engine supporting foreign key constraints and multi-statement ACID transactions, which are required for the double-entry ledger.

3. System Architecture

3.1 High-Level Architecture

The system follows a decoupled frontend/backend architecture. Vue.js runs as a Single Page Application (SPA) in the browser and communicates with Laravel exclusively through a RESTful JSON API. Laravel handles all business logic, database access, and security enforcement.



3.2 Laravel Internal Structure

- **Controllers** — handle HTTP requests, validate input, return JSON responses
- **Services** — business logic (LedgerService, TransferService, NotificationService)
- **Middleware** — CheckPermission (RBAC), BranchScope (data isolation), AuditLogger
- **Models** — Eloquent models with relationships, accessors, and scopes
- **Policies** — fine-grained authorization rules per model action
- **Migrations** — version-controlled schema files for all 17 tables
- **Seeders** — initial data: 3 branches, 10 roles, 50+ permissions, 1 Super Admin

4. Database Design

The system uses 17 MySQL tables organized in strict dependency order. MySQL InnoDB engine is required for foreign key enforcement and multi-statement ACID transactions.

4.1 All Database Tables

# Table Name	Key Columns	Purpose
1. <code>branches</code>	<code>id</code> , <code>branch_name</code> , <code>city</code> , <code>address</code> , <code>phone</code> , <code>manager_id</code> FK, <code>status</code> , <code>opened_at</code>	All bank branches. Every record in the system references this table.
2. <code>roles</code>	<code>id</code> , <code>role_name</code> , <code>tier</code> , <code>description</code> , <code>txn_limit</code> , <code>branch_restricted</code>	10 staff roles with transaction limits and branch isolation flag.
3. <code>permissions</code>	<code>id</code> , <code>permission_key</code> , <code>module</code> , <code>description</code>	Granular permission keys e.g. <code>cash.deposit</code> , <code>loan.approve</code> .
4. <code>role_permissions</code>	<code>role_id</code> FK, <code>permission_id</code> FK	Pivot table mapping roles to their allowed permissions (RBAC core).
5. <code>staff</code>	<code>id</code> , <code>full_name</code> , <code>email</code> , <code>password_hash</code> , <code>role_id</code> FK, <code>branch_id</code> FK, <code>status</code> , <code>created_at</code>	All bank staff. Linked to one role and one branch.
6. <code>customers</code>	<code>id</code> , <code>full_name</code> , <code>national_id</code> , <code>phone</code> , <code>address</code> , <code>kyc_status</code> , <code>home_branch_id</code> FK, <code>created_at</code>	Bank customers. Separate from staff.
7. <code>account_types</code>	<code>id</code> , <code>type_name</code> , <code>interest_rate</code> , <code>min_balance</code> , <code>overdraft_allowed</code> , <code>withdrawal_limit</code> , <code>is_active</code>	Configurable account types: Saving, Current, Kaftoon etc.
8. <code>accounts</code>	<code>id</code> , <code>customer_id</code> FK, <code>account_type_id</code> FK, <code>home_branch_id</code> FK, <code>account_number</code> , <code>status</code> , <code>opened_at</code>	Customer accounts. balance is always calculated from <code>ledger_entries</code> .
9. <code>transactions</code>	<code>id</code> , <code>reference</code> (unique), <code>type</code> , <code>status</code> , <code>amount</code> , <code>initiated_by</code> FK, <code>branch_id</code> FK, <code>description</code> , <code>created_at</code>	Every financial event. Status: pending, success, failed, reversed, awaiting_approval.
10. <code>ledger_entries</code>	<code>id</code> , <code>transaction_id</code> FK, <code>account_id</code> FK, <code>branch_id</code> FK, <code>entry_type</code> (debit/credit), <code>amount</code> , <code>balance_after</code> , <code>created_at</code>	CORE TABLE. Double-entry ledger. Append-only — never updated or deleted.
11. <code>pending_approvals</code>	<code>id</code> , <code>transaction_id</code> FK, <code>requested_by</code> FK, <code>approver_role_id</code> FK, <code>status</code> , <code>decided_by</code> FK, <code>decided_at</code> , <code>created_at</code>	Approval queue for transactions exceeding staff limits.
12. <code>inter_branch_clearing</code>	<code>id</code> , <code>from_branch_id</code> FK, <code>to_branch_id</code> FK, <code>transaction_id</code> FK, <code>amount</code> , <code>status</code> , <code>settled_at</code>	Tracks cross-branch transfers through the central clearing account.

13. <code>loans</code>	<code>id</code> , <code>customer_id</code> FK, <code>account_id</code> FK, <code>amount</code> , <code>interest_rate</code> , <code>term_months</code> , <code>status</code> , <code>approved_by</code> FK, <code>disbursed_at</code>	Loan records. Status: applied, under_review, approved, disbursed, closed, defaulted.
14. <code>loan_repayments</code>	<code>id</code> , <code>loan_id</code> FK, <code>due_date</code> , <code>amount</code> , <code>paid_at</code> , <code>ledger_entry_id</code> FK, <code>status</code>	Monthly repayment schedule linked to ledger entries.
15. <code>kyc_documents</code>	<code>id</code> , <code>customer_id</code> FK, <code>doc_type</code> , <code>file_path</code> , <code>verified_by</code> FK, <code>status</code> , <code>uploaded_at</code>	KYC verification documents per customer.
16. <code>notifications</code>	<code>id</code> , <code>recipient_id</code> , <code>recipient_type</code> , <code>channel</code> (sms/email), <code>message</code> , <code>sent_at</code> , <code>status</code>	Log of all notifications sent to customers and staff.
17. <code>staff_audit_log</code>	<code>id</code> , <code>staff_id</code> FK, <code>action</code> , <code>permission_used</code> , <code>target_table</code> , <code>target_id</code> , <code>ip_address</code> , <code>result</code> , <code>created_at</code>	Immutable log of every staff action. Append-only.

4.2 Critical Migration Order

Laravel migrations must run in this exact order due to foreign key dependencies:

1. `create_branches_table`
2. `create_roles_table`
3. `create_permissions_table`
4. `create_role_permissions_table`
5. `create_staff_table`
6. `create_customers_table`
7. `create_account_types_table`
8. `create_accounts_table`
9. `create_transactions_table`
10. `create_ledger_entries_table`
11. `create_pending_approvals_table`
12. `create_inter_branch_clearing_table`
13. `create_loans_table`
14. `create_loan_repayments_table`
15. `create_kyc_documents_table`
16. `create_notifications_table`
17. `create_staff_audit_log_table`

Ledger Entries — Most Important Table

`entry_type`: ENUM("debit", "credit") — only two possible values

`amount`: DECIMAL(15,2) — supports amounts up to 999 trillion with 2 decimal places

`balance_after`: DECIMAL(15,2) — snapshot of balance at time of entry (for quick display)

INDEX on (`account_id`, `entry_type`) — critical for fast balance calculations

NEVER updated or deleted. Corrections are made through reversal entries only.

5. Roles & Permissions (RBAC)

The system implements Role-Based Access Control (RBAC) using a database-driven permission model. Permissions are not hardcoded in application logic — they are stored as rows in the permissions table and assigned to roles via the role_permissions pivot table. Adding or revoking a permission requires only a database change, not a code deployment.

5.1 The 10 Staff Roles

Role	Tier	Scope	Txn Limit	Responsibilities
Super Admin	System	None	Unlimited	Full system control, all branches, creates branch managers
System Admin	System	None	N/A	Technical maintenance, backups, server health — no financial access
Branch Manager	Branch	Own branch	Unlimited (branch)	Full branch authority, approves high-value transactions, views audit logs
Operational Manager	Branch	Own branch	High	Monitors daily operations, coordinates staff, prepares daily reports
Teller Supervisor	Branch	Own branch	Above teller limit	Approves teller transactions above limit, handles exceptions
Bank Teller	Front-line	Own branch	\$5,000/txn	Cash deposits, withdrawals, transfers within limit
Customer Care Officer	Front-line	Own branch	N/A	Customer registration, KYC collection, account opening
Loan Officer	Specialist	Own branch	Approve up to \$50K	Loan applications, approval, repayment schedules
Compliance Officer	Specialist	All branches (read)	N/A	KYC verification, AML monitoring, account flagging
Internal Auditor	Oversight	All branches (read-only)	N/A	Read-only access to all transactions, ledger, audit logs

5.2 Role Hierarchy

- Super Admin → Branch Manager → Operational Manager → Teller Supervisor → Bank Teller
- Super Admin → Branch Manager → Loan Officer
- Super Admin → Branch Manager → Customer Care Officer
- Super Admin → Compliance Officer (cross-branch read access)
- Super Admin → Internal Auditor (read-only, all branches)
- Super Admin → System Administrator (infrastructure only, no financial access)

5.3 Permission Keys Reference

Permission Key	Module	Assigned To	Description
cash.deposit	Transactions	Bank Teller	Process cash deposits to customer accounts
cash.withdraw	Transactions	Bank Teller	Process cash withdrawals from customer accounts
transfer.internal	Transactions	Bank Teller, Transfer Officer	Transfer between accounts within the bank
transfer.external	Transactions	Transfer Officer	Send transfers to external banks
transfer.reverse	Transactions	Teller Supervisor, Branch Manager	Reverse a failed or erroneous transfer
teller.approve	Approvals	Teller Supervisor	Approve teller transactions above daily limit
txn.approve.high	Approvals	Branch Manager	Approve high-value transactions escalated from supervisor
account.open	Accounts	Customer Care Officer	Initiate new account opening process
account.freeze	Accounts	Branch Manager, Compliance Officer	Freeze suspicious or inactive accounts
account.close	Accounts	Branch Manager	Close customer accounts (with approval log)
customer.create	Customers	Customer Care Officer	Register new customers in the system
kyc.collect	Compliance	Customer Care Officer	Upload and collect KYC documents
kyc.verify	Compliance	Compliance Officer	Verify and approve KYC submissions
aml.monitor	Compliance	Compliance Officer	View and flag suspicious transaction patterns
loan.apply	Loans	Customer (via portal)	Submit loan application
loan.review	Loans	Loan Officer	Review and assess loan applications
loan.approve	Loans	Loan Officer (under \$50K), Branch Manager	Approve or reject loan applications
ledger.read	Ledger	Accounting Officer, Auditor, Branch Manager	View ledger entries and account balances
ledger.reconcile	Ledger	Accounting Officer	Perform trial balance and reconciliation checks
report.branch	Reports	Branch Manager, Operational Manager	Generate branch-level financial reports
report.system	Reports	Super Admin	Generate consolidated cross-branch reports

<code>audit.read</code>	Audit	Internal Auditor, Branch Manager (own branch)	Read staff audit log entries
<code>staff.manage</code>	Staff	Branch Manager (own branch), Super Admin	Create and manage staff accounts
<code>branch.manage</code>	Branches	Super Admin	Create and configure bank branches
<code>system.config</code>	System	Super Admin, System Admin	Access system-level configuration

5.4 RBAC Enforcement in Laravel

Every protected API route is wrapped with the `CheckPermission` middleware. The middleware queries the `role_permissions` table using the authenticated staff member's `role_id` and the required permission key. If no matching row exists, the request is blocked with HTTP 403 and the attempt is recorded in the `staff_audit_log`.

How Permission Check Works (Laravel Middleware)

1. Request arrives with Bearer token in Authorization header
2. Sanctum validates token → loads authenticated Staff model
3. `CheckPermission` middleware receives required permission key from route definition
4. Query: `SELECT 1 FROM role_permissions JOIN permissions WHERE staff.role_id AND permission_key = ?`
5. Row found → ALLOW, continue to Controller
6. No row found → INSERT into `staff_audit_log` (result: denied) → return HTTP 403

6. Multi-Branch Architecture

The system supports an unlimited number of branches. All branches share a single central MySQL database — there is no per-branch database or replication. Branch isolation is enforced at the application layer through the BranchScope middleware and the `branch_id` column present on every relevant table.

6.1 Core Design Principles

- Single central database — all branches read and write to the same MySQL instance
- Every record carries `branch_id` — accounts, transactions, ledger entries, staff are all tagged
- BranchScope middleware auto-appends `WHERE branch_id = ?` to all queries for branch-restricted staff
- Super Admin and Internal Auditor have no `branch_id` restriction — they see all data
- Adding a new branch requires only inserting one row into the branches table — no code change

6.2 Customer Visiting Any Branch

A customer whose home branch is Branch A (Mogadishu) can walk into Branch B (Garoowe) and be served. Because all branches share one database, the Garoowe cashier's query returns the customer's account instantly. The resulting transaction is recorded with `branch_id = Garoowe`, while the account's `home_branch_id` remains Mogadishu.

6.3 Inter-Branch Transfer — 4 Ledger Entries

When a customer in Branch A transfers money to a customer in Branch B, the system uses a central inter-branch clearing account to ensure each branch's books balance independently.

Step	Account	Branch	Entry	Meaning
1	Sender account (Fatima)	Branch A — Mogadishu	DEBIT	\$1,000 leaves Fatima's account
2	Inter-branch clearing	Branch A — Mogadishu	CREDIT	Branch A books balance (net zero)
3	Inter-branch clearing	Branch B — Garoowe	DEBIT	Branch B books balance (net zero)
4	Recipient account (Omar)	Branch B — Garoowe	CREDIT	\$1,000 arrives in Omar's account

All 4 entries are written inside a single `DB::transaction()`. If any entry fails, all 4 are rolled back and no money moves.

6.4 Branch Data Isolation Rules

- Branch Manager — sees only their branch's accounts, staff, transactions, and reports
- Bank Teller — sees only their own transactions and branch customers
- Compliance Officer — read access across all branches for AML monitoring
- Internal Auditor — read-only access to all branches, all data
- Super Admin — unrestricted access to all branches for system administration

7. General Ledger (GL) System

The General Ledger is the financial backbone of the entire banking system. It is built on the principle of double-entry bookkeeping, a method used in every bank worldwide since the 15th century. Every single movement of money is recorded as two equal and opposite entries — a debit and a credit — ensuring perfect mathematical balance at all times.

7.1 The Three Laws of the Ledger

Law 1 — Every Transaction Creates Exactly Two Entries

A DEBIT entry (money leaves one account) and a CREDIT entry (money arrives in another). Both entries carry the same transaction_id, linking them permanently.

If a DEBIT exists without a matching CREDIT in the database, money has disappeared — the system can detect this automatically.

Law 2 — Ledger Entries Are Append-Only (Never Updated or Deleted)

Once written, a ledger_entry row is permanent. No UPDATE or DELETE is ever executed on this table.

If a mistake is made, a reversal entry is written — a new pair of entries that mathematically cancels the original.

This preserves complete financial history and satisfies regulatory compliance requirements.

Law 3 — Balance Is Always Calculated, Never Stored

The accounts table does NOT have a balance column that gets updated on every transaction. Balance is always computed on demand: SUM(credits) - SUM(debits) for a given account_id.

This means the balance is always 100% accurate regardless of crashes, failures, or partial writes.

Formula: current_balance = SUM(amount WHERE entry_type=credit) - SUM(amount WHERE entry_type=debit)

7.2 Ledger Entry Types by Transaction

Transaction Type	DEBIT Account	CREDIT Account	Notes
Cash Deposit	Bank Cash Account	Customer Account	Cash received at teller counter
Cash Withdrawal	Customer Account	Bank Cash Account	Cash paid out at teller counter
Internal Transfer	Sender Account	Recipient Account	Same bank, same or different branch
Inter-Branch Transfer	Sender + IB Clearing	IB Clearing + Recipient	4 entries: 2 per branch (see Section 6.3)

Loan Disbursement	Bank Loan Account	Customer Account	Approved loan credited to customer
Loan Repayment	Customer Account	Bank Loan Account	Monthly repayment debited from customer
Interest Credit	Bank Interest Expense	Customer Account	Monthly interest on Saving accounts
Fee Debit	Customer Account	Bank Fee Income	Service charges or maintenance fees
Transfer Reversal	Recipient Account	Sender Account	Reversal entries exactly cancel original

7.3 Atomic Transaction Implementation in Laravel

Every multi-entry ledger operation is wrapped inside Laravel's `DB::transaction()` closure. MySQL automatically rolls back all statements if any exception is thrown before the closure completes.

```
// LedgerService.php - Transfer between two accounts
public function transfer(Account $from, Account $to, float $amount, Staff $staff):
Transaction
{
    return DB::transaction(function () use ($from, $to, $amount, $staff) {

        // Step 1: Create parent transaction record
        $txn = Transaction::create([
            'reference'    => 'TXN-' . strtoupper(uniqid()),
            'type'         => 'transfer',
            'status'       => 'pending',
            'amount'       => $amount,
            'initiated_by' => $staff->id,
            'branch_id'    => $staff->branch_id,
        ]);

        // Step 2: Check sufficient balance
        if ($from->balance < $amount) {
            throw new InsufficientFundsException('Balance too low');
        }

        // Step 3: Write DEBIT entry (money leaves source)
        LedgerEntry::create([
            'transaction_id' => $txn->id,
            'account_id'     => $from->id,
            'branch_id'      => $from->home_branch_id,
            'entry_type'     => 'debit',
            'amount'         => $amount,
            'balance_after'  => $from->balance - $amount,
        ]);

        // Step 4: Write CREDIT entry (money arrives at destination)
        LedgerEntry::create([
            'transaction_id' => $txn->id,
            'account_id'     => $to->id,
            'branch_id'      => $to->home_branch_id,
            'entry_type'     => 'credit',
            'amount'         => $amount,
            'balance_after'  => $to->balance + $amount,
        ]);
    });
}
```

```
// Step 5: Mark SUCCESS and return
$txn->update(['status' => 'success']);
return $txn;

// If ANY line above throws an exception,
// MySQL rolls back ALL changes automatically.
// No ledger entries are saved. No money moves.
});
}
```

7.4 Balance Calculation in Laravel (Eloquent Accessor)

```
// Account.php Model - Balance is never stored, always calculated
public function getBalanceAttribute(): float
{
    $credits = $this->ledgerEntries()
        ->where('entry_type', 'credit')
        ->sum('amount');

    $debits = $this->ledgerEntries()
        ->where('entry_type', 'debit')
        ->sum('amount');

    return (float) ($credits - $debits);
}

// Usage - always accurate, even after failed transactions:
$account->balance // returns correct figure from ledger history
```

8. Transaction & Approval Workflow

8.1 Transaction Status Lifecycle

Status	Description
PENDING	Transaction created but not yet processed. Starting state for all transactions.
AWAITING_APPROVAL	Amount exceeds staff limit. Moved to pending_approvals table for supervisor/manager action.
SUCCESS	All ledger entries committed. Money successfully moved. Final state.
FAILED	An error occurred during processing. DB::transaction() rolled back. No ledger entries saved.
REVERSED	A previously successful transaction was cancelled. Reversal ledger entries written.

8.2 Approval Escalation Flow

- Teller initiates transaction → system checks amount against role `txn_limit`
- Amount within limit → process immediately, write ledger entries, status = SUCCESS
- Amount exceeds Teller limit → insert into `pending_approvals` (`approver_role` = Teller Supervisor)
- Teller Supervisor reviews queue → approves or rejects
- Amount exceeds Supervisor limit → escalate again (`approver_role` = Branch Manager)
- Branch Manager approves → `LedgerService` processes the transaction
- Any rejection at any level → transaction status = FAILED, customer notified

8.3 Failed Transaction Detection

Staff can detect any stuck or failed transactions using the following query. Any transaction in PENDING status for more than 60 seconds is flagged for investigation:

Monitoring Query for Stuck Transactions

```
SELECT * FROM transactions
WHERE status = 'pending'
AND created_at < NOW() - INTERVAL 60 SECOND
ORDER BY created_at ASC;
```

The Accounting Officer runs this daily as part of ledger reconciliation.
Any result here means a transaction is stuck and requires manual investigation.

9. API Endpoints Reference

All API endpoints are prefixed with `/api` and return JSON. Protected routes require a Bearer token from Laravel Sanctum in the Authorization header. The Permission column shows the `permission_key` the authenticated staff must have in their `role_permissions` to access the route.

Method	Endpoint	Permission	Description
POST	/api/login	None	Authenticate staff or customer, return Sanctum token
POST	/api/logout	auth:sanctum	Revoke current token
GET	/api/me	auth:sanctum	Return authenticated user with role, branch, permissions
GET	/api/accounts/{id}	auth:sanctum	Get account details and calculated balance
GET	/api/accounts/{id}/ledger	ledger.read	Get paginated ledger entries for account
POST	/api/transactions/deposit	cash.deposit	Process cash deposit, write debit+credit entries
POST	/api/transactions/withdraw	cash.withdraw	Process cash withdrawal, write entries
POST	/api/transactions/transfer	transfer.internal	Transfer between accounts (internal or inter-branch)
POST	/api/transactions/{id}/reverse	transfer.reverse	Write reversal entries for a transaction
GET	/api/approvals	teller.approve	List pending approval requests for this staff role
POST	/api/approvals/{id}/approve	teller.approve	Approve a pending transaction
POST	/api/approvals/{id}/reject	teller.approve	Reject a pending transaction
POST	/api/customers	customer.create	Register a new customer
POST	/api/customers/{id}/kyc	kyc.collect	Upload KYC documents for customer
PUT	/api/customers/{id}/kyc/verify	kyc.verify	Mark KYC documents as verified
POST	/api/accounts	account.open	Open new account for customer
PUT	/api/accounts/{id}/freeze	account.freeze	Freeze a customer account
POST	/api/loans	loan.review	Create loan application
PUT	/api/loans/{id}/approve	loan.approve	Approve a loan application

GET	/api/reports/branch	report.branch	Get branch daily/monthly report
GET	/api/reports/system	report.system	Get consolidated system-wide report
GET	/api/audit-log	audit.read	Get filtered staff audit log
POST	/api/staff	staff.manage	Create new staff account
POST	/api/branches	branch.manage	Create new bank branch

10. Security Design

Security Layer	Implementation
Password Hashing	All passwords hashed with bcrypt (cost factor 12) before storage. Plain-text passwords are never stored or logged.
API Authentication	Laravel Sanctum issues per-session bearer tokens. Tokens are stored hashed in <code>personal_access_tokens</code> . Logout revokes the token immediately.
Role-Based Access	<code>CheckPermission</code> middleware on every protected route. Permission check is a database query — not a code condition. Roles can be changed without redeploying.
Branch Isolation	<code>BranchScope</code> middleware auto-injects <code>WHERE branch_id = ?</code> into all Eloquent queries for branch-restricted staff. Cannot be bypassed by the frontend.
Input Validation	All request inputs validated using Laravel Form Request classes before reaching controllers. SQL injection is prevented by Eloquent parameterized queries.
Audit Logging	Every API call by a staff member — including 403 blocks — is recorded in <code>staff_audit_log</code> with IP address, timestamp, and result. Table is append-only.
HTTPS Only	All communication between Vue.js frontend and Laravel API must use HTTPS in production. HTTP requests are redirected automatically.
Token Expiry	Sanctum tokens expire after configurable time (recommended: 8 hours for staff sessions). Expired tokens are rejected and the user must re-authenticate.
Immutable Ledger	No UPDATE or DELETE is permitted on <code>ledger_entries</code> table. MySQL user for the application has INSERT and SELECT privileges only on this table.

11. Build Roadmap

Phase	Weeks	Focus	Key Deliverables
Phase 1	Week 1–2	Foundation & Database	- Create Laravel 12 project, install Sanctum, configure MySQL .env - Write all 17 migration files in dependency order, run <code>php artisan migrate</code> - Create Seeders: 3 branches, 10 roles, 25 permissions, 1 Super Admin - Create all Eloquent Models with relationships (<code>hasMany</code>, <code>belongsTo</code>, <code>belongsToMany</code>) - Create Vue 3 project with Pinia + Vue Router, install Tailwind CSS and Axios - Set up 3 route groups (<code>/admin</code>, <code>/staff</code>, <code>/customer</code>) with navigation guards
Phase 2	Week 3	Authentication & RBAC	- Build POST <code>/api/login</code> and POST <code>/api/logout</code> with Sanctum token management - Implement <code>CheckPermission</code> middleware — RBAC enforcement on all routes - Implement <code>BranchScope</code> middleware — auto-filter queries by <code>staff branch_id</code> - Build Pinia auth store: token, user, permissions array, <code>branch</code>, <code>can()</code> helper - Build Vue Login page with role-based redirect after successful authentication - Build Vue 403 page and Axios interceptors for global error handling
Phase 3	Week 4–6	Core Banking Engine	- Build <code>LedgerService::transfer()</code> with <code>DB::transaction()</code> wrapping all entries - Implement Account balance Eloquent accessor — <code>SUM credits minus SUM debits</code> - Build deposit, withdrawal, and transfer API endpoints with limit checks - Implement <code>pending_approvals</code> workflow: create, escalate, approve, reject - Build inter-branch transfer with 4 ledger entries and clearing account - Implement <code>AuditLogger</code> middleware — auto-log all staff actions post-response
Phase 4	Week 7–9	All Staff UI Screens	- Build role-aware Sidebar component using <code>auth.can()</code> permission checks - Build Teller Desk view: deposit/withdraw/transfer forms with live balance - Build Approval Queue view for Teller Supervisor and Branch Manager - Build Customer Care screens: register customer, upload KYC, open account - Build Compliance Officer screen: KYC verification, AML flagging

			• Build Accounting Officer screen: ledger view, reconciliation, trial balance • Build Super Admin dashboard: all branches, total assets, staff management
Phase 5	Week 10–11	Advanced Features	• Build Loan module: apply, review, approve, disburse via ledger, repayment schedule • Implement notification system using Laravel Queues and Jobs • Build branch and consolidated financial reports • Add PDF export using barryvdh/laravel-dompdf package • Add charts to dashboards using Chart.js or ApexCharts
Phase 6	Week 12–14	Testing & Documentation	• Test all API endpoints in Postman, export collection as project deliverable • Test RBAC: confirm every role blocked from unauthorized routes • Test atomic rollback: force exception inside DB::transaction(), verify ledger clean • Test inter-branch transfer: confirm all 4 entries balanced • Finalize ERD diagram, API documentation, architecture diagram, and project report